Exercise II, Theory of Computation 2026

These exercises are for your own benefit. Feel free to collaborate and share your answers with other students. Solve as many problems as you can and ask for help if you get stuck for too long. Problems marked * are more difficult but also more fun :).

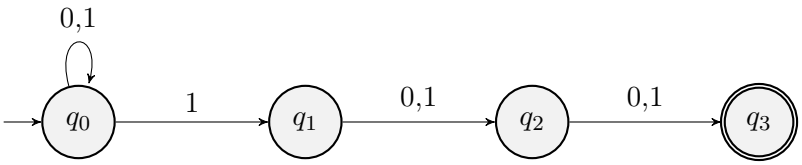
These problems are taken from various sources at EPFL and on the Internet, too numerous to cite individually.

1 For both of the following languages over $\Sigma = \{0, 1\}$, construct an NFA which recognizes it.

1a $\{w \mid w$ has a 1 in the third to last position$\}$
1b $\{w \mid w$ contains four consecutive symbols that are the same$\}$

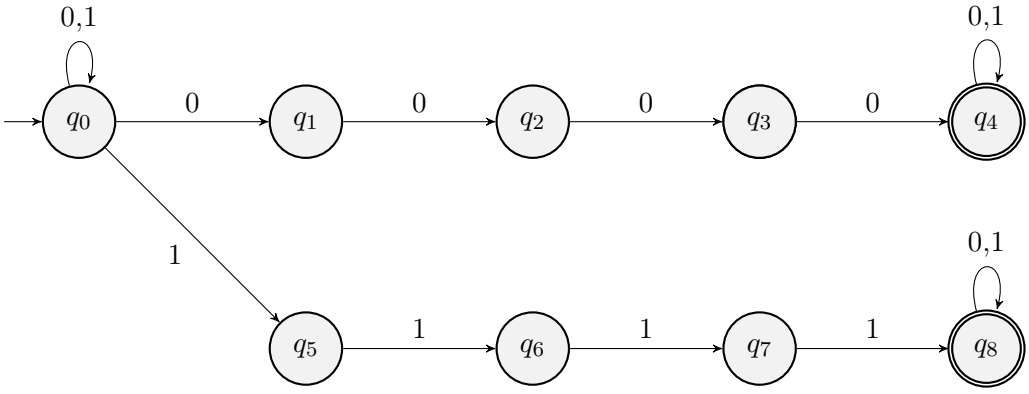
Solution:

1a The language is accepted by the NFA below.



The intuition behind the above construction is as follows: The automaton ignores all symbols of the input until at some point it "guesses" that there are only three more symbols left. Then it requires the next symbol to be a 1, followed by exactly two arbitrary symbols. We see that a string has a "chance" of being accepted by this procedure if and only if it is of the desired form.

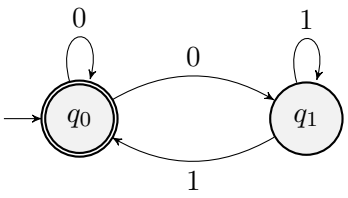
1b The language is accepted by the NFA below.



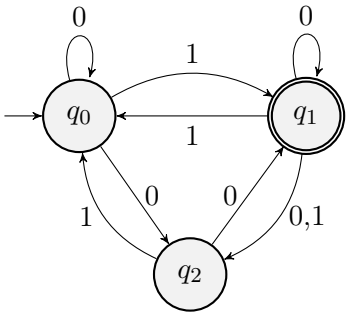
The intuition here is as follows: The automaton ignores all symbols of the input until at some point it "guesses" that the next four symbols are all the same, and if they are 0s or 1s. Then it requires the next four symbols to be consistent with that guess, after which it ignores the rest of the input. Again, a string has a "chance" of being accepted by this procedure if and only if it is of the desired form.

2 For both of the NFA over $\Sigma = \{0, 1\}$ below, construct a DFA that accepts the same language. First, write down the subset construction and then remove any states that are not reachable.

2a



2b



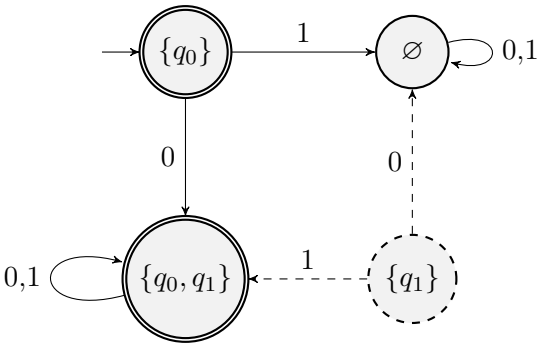
Solution: We first recall the strategy of constructing an equivalent DFA from a given NFA seen in class: The states of the DFA will be the subsets of states of the NFA. The transition function of the DFA will be constructed from the transition function of the NFA in such a way that the following holds: After heaving read an initial part of the input, we are in the DFA state which corresponds to the set of all possible NFA states we could be in, having read the same partial input. The starting and accepting states are chosen accordingly.

2a This NFA has only two states, q_0 and q_1, so our DFA will consist of the four states

$$\varnothing, \{q_0\}, \{q_1\} \text{ and } \{q_0, q_1\},$$

where $\{q_0\}$ is the starting state. The two states $\{q_0\}$ and $\{q_0, q_1\}$ will be the accepting states, as they both contain the original accepting state q_0. The transition function and state diagram of the DFA are given below.

	0	1
$\varnothing$	$\varnothing$	$\varnothing$
$\{q_0\}$	$\{q_0, q_1\}$	$\varnothing$
$\{q_1\}$	$\varnothing$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1\}$

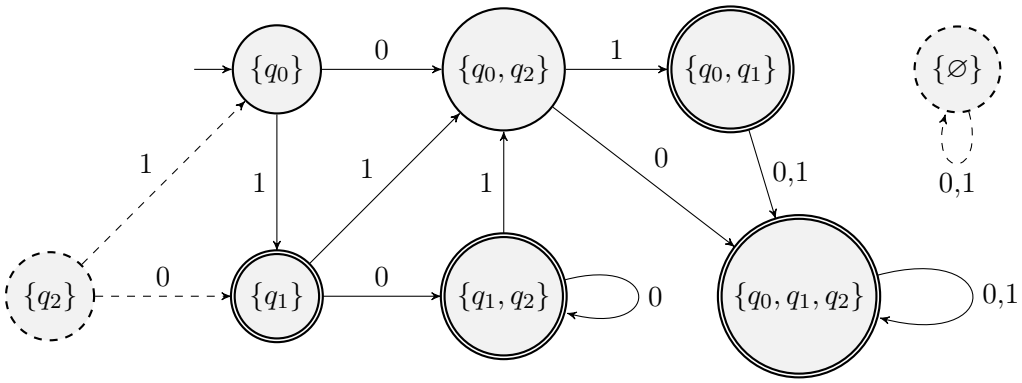


We observe that the state $\{q_1\}$ is not reachable, and thus we can remove this state together with all its outgoing arrows from the diagram, as well as the corresponding row of the table. Note that while the state $\{q_0\}$ does not appear in the above table, this state is still reachable since it is the starting state.

2b Similarly here, the states will correspond to the eight subsets of $\{q_0, q_1, q_2\}$. The starting state will be $\{q_0\}$ and the accepting states are the ones that contain the original accepting state q_1. The transition function is given by the table below.

	0	1
$\varnothing$	$\varnothing$	$\varnothing$
$\{q_0\}$	$\{q_0, q_2\}$	$\{q_1\}$
$\{q_1\}$	$\{q_1, q_2\}$	$\{q_0, q_2\}$
$\{q_2\}$	$\{q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$
$\{q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_0, q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$

The corresponding state diagram looks as follows.



We notice that the states $\varnothing$ and $\{q_2\}$ are unreachable from the start state $\{q_0\}$. Therefore we can remove these two states from the automaton.

3 Using non-determinism, give an alternative proof of the fact that the union of two regular languages is regular. Given that we already know regular languages to be closed under complementation, explain how this proof directly extends to intersections.

Solution: Given two regular languages L and L', let

$$M = (Q, \Sigma, \delta, q_0, F) \text{ and } M' = (Q', \Sigma, \delta', q_0', F')$$

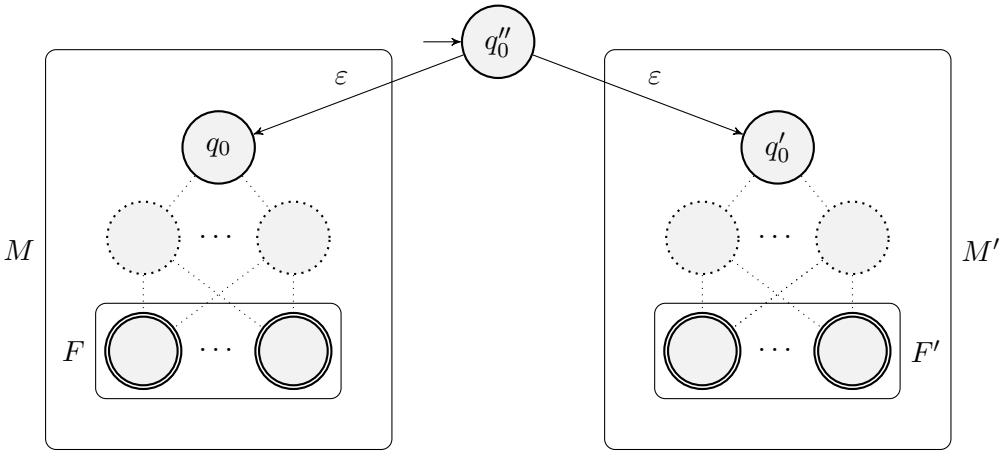
be NFA that recognise them. We now construct an NFA $N = (Q_N, \Sigma, \delta_N, q_0'', F_N)$ as follows:

- The states of N are given by $Q_N = \{q_0''\} \cup Q \cup Q'$.
- The transition function of N is given by

$$\delta_N(q, a) = \begin{cases} \delta(q, a) & \text{if } q \in Q \\ \delta'(q, a) & \text{if } q \in Q' \\ \{q_0, q_0'\} & \text{if } q = q_0'' \text{ and } a = \varepsilon \\ \varnothing & \text{else,} \end{cases}$$

  for all $q \in Q_N$ and all $a \in \Sigma \cup \{\varepsilon\}$.
- The accepting states of N are given by $F_N = F \cup F'$.

This automaton recognises the language $L \cup L'$ and is illustrated below.



The intuition is that at the very beginning, N must make a choice between L and L', and afterwards N behaves exactly like the automaton corresponding to the chosen language.

As soon as we know that regular languages are closed under the operations of union and complementation, we can use De Morgan's law: For any two regular languages L and L', the language

$$L \cap L' = \overline{(\overline{L} \cup \overline{L'})}$$

is also regular.

4 Given an NFA over some alphabet Σ, show how to construct an NFA over Σ that accepts the same language, but does not contain any ε-transitions.

Solution: Let $N = (Q, \Sigma, \delta, q_0, F)$ be an arbitrary NFA. Recall that ε-transitions allow an automaton to change its state spontaneously. The idea behind the construction is this: When N is in some state $q \in Q$, we consider all states reachable from q with any number of ε-transition, then we transit from all these states according to the transition function, given the next input symbol. With this strategy in mind, we define the *epsilon closure* of a set of states $A \subseteq Q$, given by

$$E(A) = \{q' \in Q \mid q' \text{ is reachable from some } q \in A \text{ by some number of } \varepsilon\text{-transitions}\}.$$

Note that $E(A)$ includes all elements of A since they can be reached using no transitions at all. Now we construct an NFA $N' = (Q, \Sigma, \delta', q_0, F')$ with the same set of states and the same starting state:

- For the accepting states of N' we take $F' = \{q \in Q \mid E(\{q\}) \cap F \neq \varnothing\}$.
- The transition function of N' we take to be

$$\delta'(q, a) = \bigcup_{q' \in E(\{q\})} \delta(q', a) = \{q' \in Q \mid \delta(q'', a) = q' \text{ for some } q'' \in E(\{q\})\}$$

  for all $q \in Q$ and all $a \in \Sigma$. Moreover, we set $\delta'(q, \varepsilon) = \varnothing$ for all $q \in Q$.

Observe that we chose δ' to map $Q \times \{\varepsilon\}$ to the empty set since we do not want N' to make use of ε-transitions. Moreover, the modification to the set of accepting states was necessary, since if a state can reach an accepting state with only ε-transition in N, this state should be an accepting state in N'.

We now formally prove that $L(N') = L(N)$. For any string w, let $\Delta(w) \subseteq Q$ denote the set of all states N could be in after reading the string w. Similarly define Δ' for N'.

Claim. For all strings w, we have $\Delta(w) = E(\Delta'(w))$.

We prove the claim by induction on the length of w. If w is the empty string, then by definition we have $\Delta(\varepsilon) = E(q_0)$ and $\Delta'(\varepsilon) = q_0$, as desired.

If w in non-empty, write $w = w'a$ for some string w' and $a \in \Sigma$. By our induction hypothesis we know $\Delta(w') = E(\Delta'(w'))$. By definition of the extended transition function, we have

$$\Delta(w) = \bigcup_{q \in \Delta(w')} E(\delta(q, a)) = E\left(\bigcup_{q \in E(\Delta'(w'))} \delta(q, a)\right) = E\left(\bigcup_{q \in \Delta'(w')} \delta'(q, a)\right) = E(\Delta'(w)),$$

as desired. In the second equality, we have used the inductive hypothesis, together with the fact that the epsilon closure distributes over the union.

We conclude that for all strings w,

$$w \in L(N') \iff \Delta'(w) \cap F' \neq \varnothing \iff E(\Delta'(w)) \cap F \neq \varnothing,$$

where we used the definition of F'. But now by the above claim,

$$E(\Delta'(w)) \cap F \neq \varnothing \iff \Delta(w)) \cap F \neq \varnothing \iff w \in L(N).$$

5 For a language L over some alphabet Σ, we define its *Kleene closure* as

$$L^* = \{w_1 w_2 \cdots w_k \mid k \geq 0 \text{ and } w_1, w_2, \ldots, w_k \in L\}.$$

In other words, L^* contains all strings obtainable by concatenating some (not necessarily distinct) strings from L. Note in particular that $\varepsilon \in L^*$ and $L \subset L^*$, corresponding to $k = 0, 1$.

Prove that if L is regular then L^* is also regular.

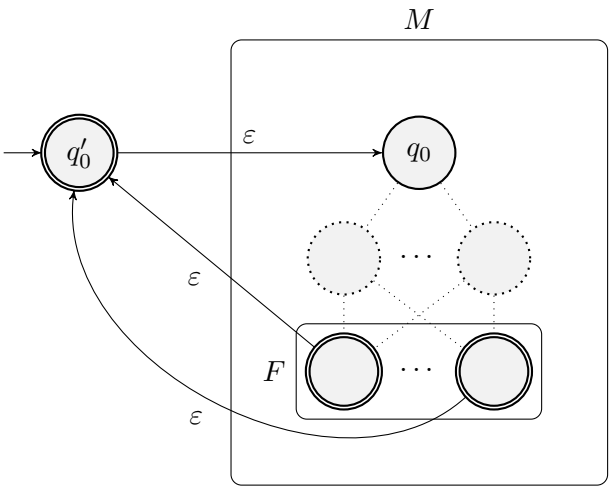
Solution: Let L be any regular language recognised by some DFA $M = (Q, \Sigma, \delta, q_0, F)$. We now construct an NFA $N = (Q', \Sigma, \delta', q_0', F')$ as follows:

- The set of states of N is given by $Q' = Q \cup \{q_0'\}$.
- The transition function δ' of N is given by

$$\delta'(q, a) = \begin{cases} \{\delta(q, a)\} & \text{if } q \in Q \\ \varnothing & \text{if } q = q_0' \end{cases} \quad \text{and} \quad \delta'(q, \varepsilon) = \begin{cases} \{q_0'\} & \text{if } q \in F \\ \{q_0\} & \text{if } q = q_0' \\ \varnothing & \text{else} \end{cases}$$

  for all $q \in Q$ and all $a \in \Sigma$.
- The accepting states of N are given by $F' = \{F \cup q_0'\}$.

In other words, we obtain N from M by simply adding ε-transitions from all states in F to q_0 and adding a new start state q_0' which is accepting and has a single ε-transition to q_0. The resulting NFA N is illustrated below.



Clearly, N accepts the empty string, and any non-empty string is accepted if and only if it can be written as the concatenation of some number of strings accepted by M, as desired.

Note that the additional state q_0' is required to guarantee that the new automaton accepts the empty string. Just making q_0 accepting does not work in general, since a path might revisit q_0 later on and accept an undesired string. This would however work if q_0 happened to be an accepting state in M already.

6* Given a language L over $\Sigma = \{0, 1\}$, define

$$L_{11} = \{xy \mid x11y \in L\}.$$

In other words, L_{11} is the language of all strings obtainable by taking some string in L and deleting two consecutive 1s from it.

Prove that if L is regular, then L_{11} is also regular.

Hint: Add transitions that simulate skipping over two 1*s. Ensure exactly one of them is used.*

Solution: Let L be an arbitrary regular language recognised by some DFA M. To prove that L_{11} is also a regular language, it suffices to create an NFA that recognises it. Before giving the technical details, we will motivate the construction.

How can we characterise an arbitrary string $w \in L_{11}$? It corresponds to a string $w' \in L$ where two consecutive 1s have been removed. In other words, w corresponds to an accepting path through the state diagram of M, where there is a single moment where we take two consecutive arrows labelled 1, but without reading any input symbols. The part of the input we read before this "jump" corresponds to the part x, while the remaining part of the input we read afterwards corresponds to y in the decomposition $w = x11y$. In terms of memory, there are two things to keep track of: The current state of M we are in, as well as whether or not we have performed our "jump" yet. Let's see how we can put this intuition into a formal construction.

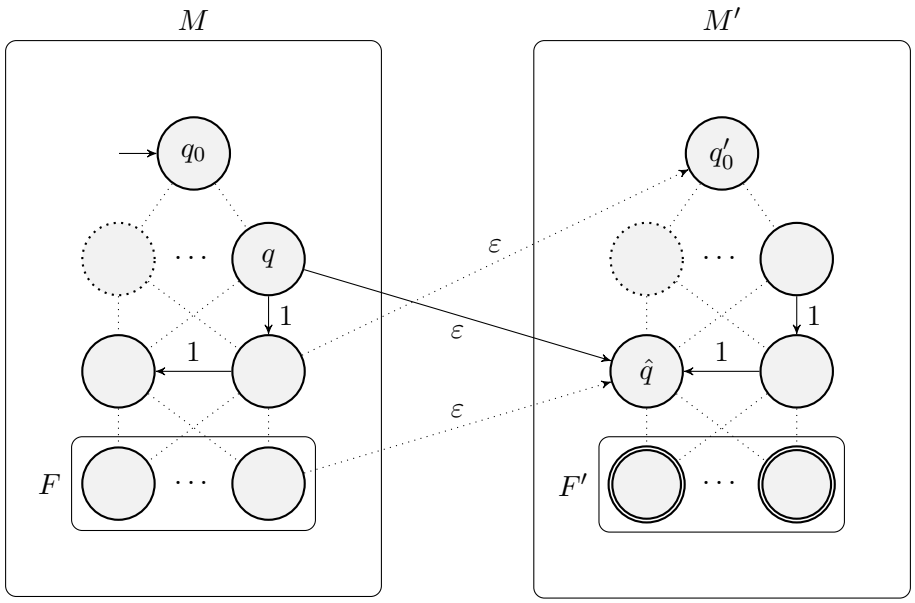
Consider a copy $M' = (Q', \Sigma, \delta', q_0', F')$ of the DFA $M = (Q, \Sigma, \delta, q_0, F)$. For any state $q \in Q$, let $\hat{q} \in Q'$ correspond to the unique state in Q which is reached from q after two consecutive arrows labelled 1. The automata M and M' will correspond to the computation before and after a "jump" which happens between some pair $(q, \hat{q})$. We now construct an NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$ with initial state q_0, given by the following:

- The states of N are given by $Q_N = Q \cup Q'$.
- The transition function of N is given by

$$\delta_N(q, a) = \begin{cases} \{\delta(q, a)\} & \text{if } q \in Q \\ \{\delta'(q, a)\} & \text{if } q \in Q' \end{cases} \quad \text{and} \quad \delta_N(q, \varepsilon) = \begin{cases} \{\hat{q}\} & \text{if } q \in Q \\ \varnothing & \text{else} \end{cases}$$

  for all $q \in Q_N$ and all $a \in \Sigma$.
- The accepting states of N are given by $F_N = F'$.

The NFA N is illustrated below.



We now prove that $L(N) = L_{11}$ by arguing inclusion in both directions. If $w \in L_{11}$, then there are strings x, y with $w = xy$ and $x11y \in L$. Since M accepts $x11y$, there is some corresponding accepting path P through M. By following the same path, the automaton N can process the string x in the part given by M and end up in a state from which there are two consecutive arrows labelled 1. Thus, by definition of δ_N, there is an ε-transition to the part given by M'. There we again follow the path P while processing y before reaching an accepting state. Hence, N accepts $w = xy$, as desired.

Conversely, let $w \in L(N)$. Thus, there must be a corresponding accepting path P through N. Since N starts from $q_0 \in Q$ and all its accepting states are in Q', we must at some point take a transition from Q to Q'. But the only such transitions are the one-way ε-transitions given by the pairs $(q, \hat{q})$, so we must make use of exactly one of them. If we now replace this transition with the corresponding pair of consecutive arrows labelled 1, we retrieve an accepting path through M of some string $x11y$ with $xy = w$ that ends in F instead of F'. We conclude that M accepts $x11y$, implying that $x11y \in L$ and therefore $w \in L_{11}$.

7* For two strings $a, b \in \Sigma$ of equal length, their *Hamming distance*, denoted by $H(a, b)$, is the number of indices at which a and b differ. Given a language L over alphabet Σ, define

$$\Gamma(L) = \{w \mid \text{there is some } w' \in L \text{ with } |w'| = |w| \text{ and } H(w, w') \leq 1\}.$$

Prove that if L is regular, then $\Gamma(L)$ is also regular.

Hint: Add transitions that simulate a wrong input symbol. Ensure at most one of them is used.

Solution: Let L be an arbitrary regular language recognised by the DFA $M = (Q, \Sigma, \delta, q_0, F)$. We will construct an NFA N that recognises $\Gamma(L)$, using a similar strategy as in the previous problem: While reading the input just as M would, the new automaton N has the option to change a single symbol of the input, after which it transitions into a disjoint section where it finished reading the input, again just as M would.

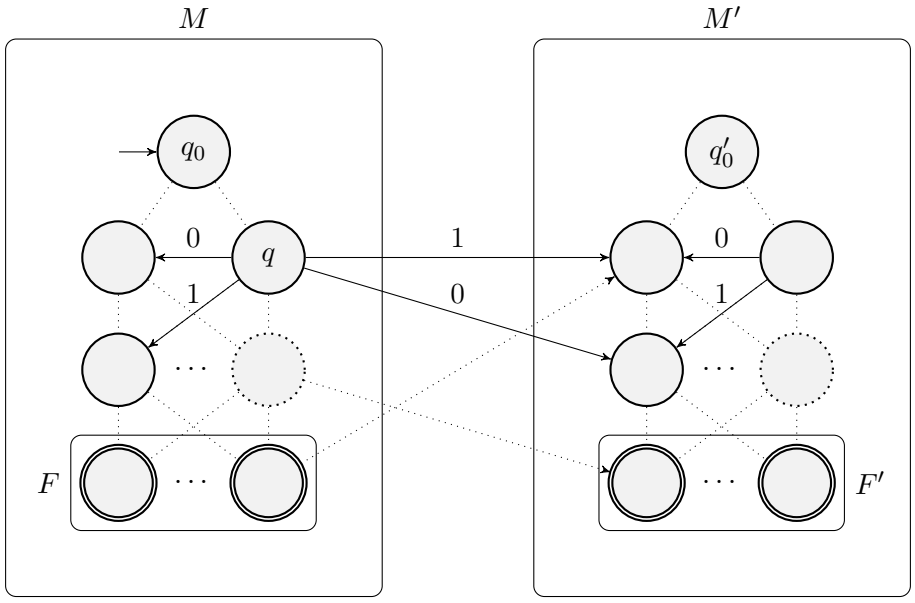
Consider a copy $M' = (Q', \Sigma, \delta', q_0', F')$ of M. For each state $q \in Q$, let $q' \in Q'$ be the corresponding state of M'. We now define the NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$ with the same start state as Q_0:

- The states of N are given by $Q_N = Q \cup Q'$.
- The transition function of N is given by

$$\delta_N(q, a) = \begin{cases} \{\delta(q, a)\} \cup \{\delta(q, b)' \mid b \in \Sigma,\ b \neq a\} & \text{if } q \in Q \\ \{\delta'(q, a)\} & \text{if } q \in Q' \end{cases} \quad \text{and} \quad \delta_N(q, \varepsilon) = \varnothing$$

  for all $q \in Q_N$ and all $a \in \Sigma$.
- The accepting states of N are given by $F_N = F \cup F'$.

Note that we included both F and F' in the accepting states since we also want to accept inputs that are at Hamming distance 0 from (i.e. equal to) a string in L. The constructed automaton N is illustrated below for the alphabet $\Sigma = \{0, 1\}$.



We again prove that $\Gamma(L) = L(N)$ by showing both inclusions: Consider any string $w \in \Gamma(L)$. If $w \in L$, then the accepting path of w through M is still an accepting path in N, thus $w \in L(N)$. Else, w has Hamming distance 1 to some string in L. in particular there are strings x, y and symbols $a \neq b \in \Sigma$ with $w = xay$ and $xby \in L$. If we now take the accepting path of xby through M and take the transition labelled a over to Q' after reading x, we get an accepting path or w through N. We conclude that $w \in L(N)$ in this case also.

Conversely, any string $w \in L(N)$ has an accepting path P through N which starts in state $q_0 \in Q$. If P terminates in F, then it must be entirely contained in Q, as there are no transitions from Q' back to Q. Thus, P is also an accepting path through M alone and we deduce $w \in L$. If P instead terminates in F', then there must be a unique point where we transition from a state $q \in Q$ to Q'. Let the label of this transition be $a \in \Sigma$ and by definition of δ_N, it must lead to some state $\delta(q, b)' \in Q'$ where $b \in \Sigma$ with $a \neq b$. If we now change the symbol of the input at this position from a to b, we recover a corresponding accepting path through M alone. The input it accepts has Hamming distance exactly 1 from w, implying that $w \in \Gamma(L)$.

Alternatively, we could have only made the states from F' accepting, and in turn for all $q \in Q$ and $a \in \Sigma$ adding another transition $\delta(q, a)' \in \delta_N(q, a)$ corresponding to going from Q to Q' without making any changes to the string.

8 In lectures, we have seen that if L is regular, then so is its complement $\overline{L}$. This was done by taking a DFA that recognises L and changing accepting states for non-accepting states and vice versa. Could we do the same with an NFA that recognises L instead? Explain your answer.

Solution: No, the same construction does not work if we instead use an NFA. While the automaton we would end up with is a valid NFA, it will in general not accept the complementary language, due to the asymmetric role of accepting and rejecting states in non-deterministic automata:

Let N be an NFA recognising L. A string w is in $\overline{L}$ if and only if N rejects w. By definition of a non-deterministic automaton, this happens if and only if

> "**All** possible paths though M corresponding to w end in a non-accepting state".

However, the proposed construction would accept a given input string w if and only if

> "There **exists** a path through M corresponding to w that ends in a non-accepting state".

In general, these two characterisations are clearly different and it is not hard to find a concrete example where they disagree (take for instance the NFA given in problem **2a**).